

Gradient Boosting in C++

1. Problem Setting

Suppose we observe pairs (x, y) produced by some unknown relationship $f : \mathcal{X} \rightarrow \mathcal{Y}$, where $\mathcal{X} \subseteq \mathbb{R}^p$ is the *feature space* and $\mathcal{Y} \subseteq \mathbb{R}$. A coordinate of $x \in \mathcal{X}$ is called a *feature*; for example, if x describes a house, its coordinates might encode floor area, number of rooms, and year built.

Our goal is to construct an approximation $\hat{f} : \mathcal{X} \rightarrow \mathcal{Y}$ from a finite training sample

$$D = \{(x_1, y_1), \dots, (x_N, y_N)\} \subset \mathcal{X} \times \mathcal{Y},$$

so that $\hat{f}(x) \approx f(x)$ on inputs not necessarily seen during training. Gradient boosting is one method of producing such an $\hat{f}$. It builds $\hat{f}$ as a sum of many small, structurally simple functions called *weak learners* (here: shallow decision trees), each fitted to correct the errors of those preceding it. The remainder of this document develops the algorithm and a C++ implementation.

2. Data Layer

Statistical algorithms require a concrete way to store the training data. The matrix $X \in \mathbb{R}^{N \times p}$ has one row per observation x_i and one column per feature; the vector $Y \in \mathbb{R}^N$ holds the corresponding targets y_i . Here N is the number of observations and p the number of features.

2.1. Data Matrix

X is stored as a flat `std::vector<double>` with row-major index mapping $\text{idx}(i, j) = i \cdot \text{cols} + j$ (here $\text{cols} = p$):

Matrix dimensions	$N \times p$	<code>size_t rows, cols;</code>
Elements	$x_{i,j} \in \mathbb{R}$	<code>std::vector<double> data;</code>
Index mapping	$\text{idx}(i, j) \rightarrow \text{data position}$	<code>index = i * cols + j;</code>

```
#include <vector>

struct Matrix {
    size_t rows;
    size_t cols;
    std::vector<double> data;

    Matrix(size_t r, size_t c) : rows(r), cols(c), data(r * c, 0.0) {}

    double& at(size_t row, size_t col) {
        return data[row * cols + col];
    }

    const double& at(size_t row, size_t col) const {
        return data[row * cols + col];
    }
};
```

3. Supervised Learning

To choose $\hat{f}$ in a principled way we introduce a *loss function* $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$, where $\mathcal{L}(y, \hat{y})$ measures the cost of predicting $\hat{y}$ when the true value is y . The exact choice of $\mathcal{L}$ is a modelling decision, made later. Given $\mathcal{L}$, we pick $\hat{f}$ to minimize the average loss over the training sample:

$$\hat{f} = \arg \min_f \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, f(x_i))$$

3.1. L_2 Loss

For continuous regression the standard choice is the *Mean Squared Error* (MSE), which penalises predictions proportionally to their squared distance from the truth:

$$\mathcal{L}(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$$

The factor of $\frac{1}{2}$ is a convention that cancels cleanly when differentiating.

4. Decision Trees

A *decision tree* is a model that partitions $\mathcal{X}$ into finitely many disjoint regions and assigns one prediction to each. When the prediction is a real number, as here, we call it a *regression tree*. Concretely, a regression tree partitions $\mathcal{X}$ into J disjoint regions $R_1, \dots, R_J \subseteq \mathcal{X}$ with $\bigcup_{j=1}^J R_j = \mathcal{X}$, and predicts a constant c_j for every x falling into region R_j :

$$\hat{f}(x) = \sum_{j=1}^J c_j \mathbb{1}(x \in R_j)$$

To keep our notation clean, for each region R_j we write

$$I_j = \{i \in \{1, \dots, N\} : x_i \in R_j\}$$

for the *index set* of training observations falling in R_j . Sums of the form $\sum_{i \in R_j}$ in what follows are shorthand for $\sum_{i \in I_j}$.

4.1. Minimizing SSE

The goal is to find regions and constants that minimize the total Sum of Squared Errors over the training sample:

$$\text{SSE} = \sum_{j=1}^J \sum_{i \in R_j} (y_i - c_j)^2$$

For a fixed region R_j , the optimal constant is found by differentiating the inner sum with respect to c_j and setting it to zero:

$$-2 \sum_{i \in R_j} (y_i - c_j) = 0 \quad \Rightarrow \quad \hat{c}_j = \frac{1}{|I_j|} \sum_{i \in R_j} y_i$$

So $\hat{c}_j$ is simply the mean of the targets in R_j .

4.2. Greedy Recursive Partitioning

Finding the globally optimal partition is NP-hard, so instead a greedy top-down algorithm is used. A *node* of the tree corresponds to a region $R \subseteq \mathcal{X}$ and the index set $I \subseteq \{1, \dots, N\}$ of training points lying in it; the root node corresponds to all of $\mathcal{X}$ and $I = \{1, \dots, N\}$. At each node we attempt to *split* its region into two child regions, then recurse.

A split is defined by a feature index $k \in \{1, \dots, p\}$ and a threshold $s \in \mathbb{R}$, and produces two candidate child regions

$$R_1(k, s) = \{x \in R : x_k \leq s\} \quad \text{and} \quad R_2(k, s) = \{x \in R : x_k > s\},$$

with corresponding index sets $I_1(k, s)$ and $I_2(k, s)$. The algorithm selects the (k, s) pair that minimizes the combined SSE of the two children, evaluated using the optimal leaf constants $\hat{c}_1, \hat{c}_2$ for those candidates:

$$\min_{k,s} \left[\sum_{i \in I_1(k,s)} (y_i - \hat{c}_1)^2 + \sum_{i \in I_2(k,s)} (y_i - \hat{c}_2)^2 \right]$$

Writing R_m for the (fixed) region at the current node, this minimisation is equivalent to maximising the *variance reduction*

$$\Delta(k, s) = \text{Var}(R_m) - \left[\frac{|I_1(k, s)|}{|I_m|} \text{Var}(R_1(k, s)) + \frac{|I_2(k, s)|}{|I_m|} \text{Var}(R_2(k, s)) \right]$$

over (k, s) . Since $\text{Var}(R_m)$ is fixed once the node is fixed, maximising Δ is the same as minimising the weighted sum of child variances, which is what the SSE expression above encodes.

5. C++ Implementation of the Regression Tree

5.1. Node Structure

The tree is stored as a flat `std::vector<Node>`, where parent nodes reference their children by vector index. This avoids pointer-based tree structures and keeps memory contiguous.

feature_idx	int	Splitting feature k ; -1 indicates a leaf
threshold	double	Split threshold s
prediction	double	Leaf constant $\hat{c}_j$
left_child	int	Index of child where $x_{i,k} \leq s$
right_child	int	Index of child where $x_{i,k} > s$

```
struct Node {
    int feature_idx = -1;
    double threshold = 0.0;
    double prediction = 0.0;
    int left_child = -1;
    int right_child = -1;

    Node(double pred) : prediction(pred) {}

    bool is_leaf() const { return feature_idx == -1; }
};
```

5.2. $O(N \log N)$ Split Search

For each feature column k , the algorithm sorts observations by x_k in $O(N \log N)$ time. It then sweeps through the sorted order, maintaining running sums to evaluate the SSE of each candidate split in $O(1)$ per step.

The key identity, that we can use to simplify calculations is:

$$\sum_{i \in R} (y_i - \bar{y})^2 = \sum_{i \in R} y_i^2 - \frac{1}{|R|} \left(\sum_{i \in R} y_i \right)^2$$

This lets SSE be updated incrementally as observations shift from the right child to the left, rather than being recomputed from scratch. The full split search over all p features costs $O(pN \log N)$ per node.

```
struct SplitResult {
    int feature_idx = -1;
    double threshold = 0.0;
    double best_sse = std::numeric_limits<double>::infinity();
};

SplitResult find_best_split(const Matrix& X, const std::vector<double>& y, const
std::vector<int>& indices) {
    SplitResult best_split;
    size_t n = indices.size();
    if (n < 2) return best_split;

    for (size_t k = 0; k < X.cols; ++k) {
        std::vector<std::pair<double, int>> feature_vals(n);
        for (size_t i = 0; i < n; ++i) {
            feature_vals[i] = {X.at(indices[i], k), indices[i]};
        }

        std::sort(feature_vals.begin(), feature_vals.end());

        double sum_left = 0.0, sum_right = 0.0;
        double sum_sq_left = 0.0, sum_sq_right = 0.0;

        for (size_t i = 0; i < n; ++i) {
            double target = y[feature_vals[i].second];
            sum_right += target;
            sum_sq_right += target * target;
        }

        size_t n_left = 0, n_right = n;

        for (size_t i = 0; i < n - 1; ++i) {
            double target = y[feature_vals[i].second];

            sum_left += target;
            sum_sq_left += target * target;
            n_left++;

            sum_right -= target;
            sum_sq_right -= target * target;
            n_right--;

            if (feature_vals[i].first == feature_vals[i+1].first) continue;
```

```

    double sse_left = sum_sq_left - (sum_left * sum_left) / n_left;
    double sse_right = sum_sq_right - (sum_right * sum_right) / n_right;
    double total_sse = sse_left + sse_right;

    if (total_sse < best_split.best_sse) {
        best_split.best_sse = total_sse;
        best_split.feature_idx = k;
        best_split.threshold = (feature_vals[i].first +
feature_vals[i+1].first) / 2.0;
    }
}
}
return best_split;
}

```

The threshold is set to the midpoint between adjacent sorted values, so new observations can be compared cleanly against it.

5.3. Tree Construction and Stopping Criteria

The tree is built recursively. At each recursive call, `find_best_split` is invoked on the current node's index subset. If a valid split is found, two child nodes are pushed onto the `std::vector<Node>` and the function recurses on each.

Recursion stops under the following conditions:

- **Maximum depth reached:** the depth parameter equals `max_depth`.
- **Insufficient observations:** fewer than 2 samples remain at the node, making a split impossible.
- **No valid split found:** all observations share the same feature value (the split search returns `feature_idx = -1`).

When recursion stops, the active node retains its initial prediction value (set at construction to the mean of its assigned targets), and is treated as a leaf via `is_leaf()`.

```

void build_tree_recursive(const Matrix& X, const std::vector<double>& y, const
std::vector<int>& indices,
    int depth, int max_depth, std::vector<Node>& tree, int node_idx) {
    if (depth ≥ max_depth || indices.size() < 2) return;

    SplitResult best = find_best_split(X, y, indices);
    if (best.feature_idx == -1) return;

    tree[node_idx].feature_idx = best.feature_idx;
    tree[node_idx].threshold = best.threshold;

    std::vector<int> left_idx, right_idx;
    double left_sum = 0.0, right_sum = 0.0;

    for (int idx : indices) {
        if (X.at(idx, best.feature_idx) ≤ best.threshold) {
            left_idx.push_back(idx);
            left_sum += y[idx];
        } else {
            right_idx.push_back(idx);
            right_sum += y[idx];
        }
    }
}

```

```

    }

    if (!left_idx.empty()) {
        tree.push_back(Node(left_sum / left_idx.size()));
        tree[node_idx].left_child = tree.size() - 1;
        build_tree_recursive(X, y, left_idx, depth + 1, max_depth, tree,
tree[node_idx].left_child);
    }

    if (!right_idx.empty()) {
        tree.push_back(Node(right_sum / right_idx.size()));
        tree[node_idx].right_child = tree.size() - 1;
        build_tree_recursive(X, y, right_idx, depth + 1, max_depth, tree,
tree[node_idx].right_child);
    }
}

std::vector<Node> build_tree(const Matrix& X, const std::vector<double>& y, const
std::vector<int>& indices, int max_depth) {
    double sum = 0.0;
    for (int idx : indices) sum += y[idx];

    std::vector<Node> tree;
    tree.push_back(Node(sum / indices.size())); // Root node
    build_tree_recursive(X, y, indices, 0, max_depth, tree, 0);
    return tree;
}

```

5.4. Prediction

To predict for a single observation x_i , the tree is traversed from the root by following left or right branches based on threshold comparisons, until a leaf is reached:

```

double predict_single_tree(const std::vector<Node>& tree, const std::vector<double>&
x_i) {
    int curr = 0;
    while (!tree[curr].is_leaf()) {
        if (x_i[tree[curr].feature_idx] ≤ tree[curr].threshold) {
            curr = tree[curr].left_child;
        } else {
            curr = tree[curr].right_child;
        }
    }
    return tree[curr].prediction;
}

```

6. Limitations of a Single Tree

A decision tree grown to sufficient depth can perfectly fit any training dataset. This is actually a problem: a single observation changing can alter the root split, cascading into a structurally different tree. The model has high variance.

Ensemble methods address this. Gradient boosting takes a sequential additive approach: instead of growing one deep tree, it combines many shallow, constrained trees (the weak learners mentioned in §1), each of which corrects the errors of its predecessors.

7. Gradient Descent in Function Space

7.1. Classical Parameter Gradient Descent

In parametric models (e.g. linear regression, neural networks), there is a finite weight vector $\theta \in \mathbb{R}^d$ to optimize. The update rule is:

$$\theta_m = \theta_{m-1} - \eta \nabla_{\theta} \mathcal{L}(\theta_{m-1})$$

The gradient points in the direction of steepest ascent of $\mathcal{L}$; subtracting it (scaled by learning rate η) moves θ toward a minimum.

7.2. From Parameter Space to Function Space

Gradient boosting has no fixed weight vector. Instead, the object being updated is the prediction function F itself, evaluated at each training point. The current model's predictions form a vector in $\mathbb{R}^N$:

$$\hat{F} = \begin{pmatrix} F(x_1) \\ F(x_2) \\ \vdots \\ F(x_N) \end{pmatrix}$$

The gradient of the empirical loss with respect to these predictions is computed pointwise:

$$g_i = \frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F(x_i)} \Big|_{F=F_{m-1}}$$

For L_2 loss, this is:

$$\frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F(x_i)} = -(y_i - F(x_i))$$

So the negative gradient is just the raw residual:

$$-g_i = y_i - F(x_i)$$

A gradient descent step in function space then looks like:

$$F_{m(x_i)} = F_{m-1}(x_i) + \eta(y_i - F_{m-1}(x_i))$$

7.3. Generalizing with a Weak Learner

The negative gradients $-g_i$ are only defined at the N training points. To generalize to unseen x , a shallow decision tree $h_{m(x)}$ is fitted to predict $-g_i$ from x_i . This extends the gradient step to the full input space:

$$F_{m(x)} = F_{m-1}(x) + \eta h_{m(x)}$$

The ensemble built by repeating this process is gradient descent in an infinite-dimensional function space, with trees serving as the update direction at each step.

8. The Gradient Boosting Algorithm

8.1. Initialization

The algorithm begins with the constant prediction that minimizes the total loss. For L_2 , this is the global mean:

$$F_0(x) = \arg \min_c \sum_{i=1}^N \mathcal{L}(y_i, c) = \frac{1}{N} \sum_{i=1}^N y_i = \bar{y}$$

8.2. Boosting Loop (for $m = 1$ to M)

Each iteration executes the following steps.

Step 1 — Compute pseudo-residuals. For each i , evaluate the negative gradient of the loss at the current ensemble prediction:

$$r_{i,m} = -\frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F(x_i)} \Big|_{F=F_{m-1}} = y_i - F_{m-1}(x_i)$$

Step 2 — Fit a weak learner. Train a regression tree $h_{m(x)}$ on the dataset (X, r_m) , where $r_m = (r_{1,m}, \dots, r_{N,m})$.

Step 3 — Compute optimal leaf values. For each leaf region $R_{j,m}$, find the constant $\gamma_{j,m}$ minimizing the loss:

$$\gamma_{j,m} = \arg \min_{\gamma} \sum_{x_i \in R_{j,m}} \mathcal{L}(y_i, F_{m-1}(x_i) + \gamma)$$

For L_2 loss, this is the mean of the pseudo-residuals in the leaf — exactly what a regression tree already computes. Step 3 is therefore absorbed automatically into tree construction.

Step 4 — Update the ensemble. Shrink the new tree by learning rate $0 < \nu \leq 1$ and add it to the model:

$$F_{m(x)} = F_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_{j,m} \mathbb{1}(x \in R_{j,m})$$

Choosing $\nu < 1$ prevents each tree from contributing too aggressively. Small ν (e.g. 0.1) acts as regularization, reducing overfitting at the cost of requiring more trees.

9. C++ Implementation

9.1. Model Structure

The `GradientBoostingRegressor` stores the initial prediction, the learning rate, and the full ensemble of trees:

```
struct GradientBoostingRegressor {  
    int n_estimators;  
    double learning_rate;  
    int max_depth;  
    double initial_prediction;  
};
```



```

std::vector<std::vector<Node>> ensemble;

GradientBoostingRegressor(int estimators, double lr, int depth)
    : n_estimators(estimators), learning_rate(lr),
      max_depth(depth), initial_prediction(0.0) {}

void fit(const Matrix& X, const std::vector<double>& Y);
double predict(const std::vector<double>& x_i) const;
double mse(const Matrix& X, const std::vector<double>& Y) const;
};

```

9.2. Training

```

void GradientBoostingRegressor::fit(const Matrix& X, const std::vector<double>& Y) {
    size_t N = Y.size();

    double sum = 0.0;
    for (double y : Y) sum += y;
    initial_prediction = sum / N;

    std::vector<double> F_m(N, initial_prediction);
    std::vector<double> pseudo_residuals(N, 0.0);

    std::vector<int> all_indices(N);
    std::iota(all_indices.begin(), all_indices.end(), 0);

    for (int m = 0; m < n_estimators; ++m) {
        for (size_t i = 0; i < N; ++i) {
            pseudo_residuals[i] = Y[i] - F_m[i];
        }

        std::vector<Node> tree = build_tree(X, pseudo_residuals, all_indices,
max_depth);
        ensemble.push_back(tree);

        for (size_t i = 0; i < N; ++i) {
            std::vector<double> x_i(X.cols);
            for (size_t j = 0; j < X.cols; ++j) {
                x_i[j] = X.at(i, j);
            }

            double tree_pred = predict_single_tree(tree, x_i);
            F_m[i] += learning_rate * tree_pred;
        }
    }
}

```

Because `build_tree` stores the arithmetic mean of its target values in each leaf, and the optimal $\gamma_{j,m}$ for L_2 loss is also that arithmetic mean, Steps 2 and 3 are merged into a single tree-fit call.

9.3. Inference

```

double GradientBoostingRegressor::predict(const std::vector<double>& x_i) const {
    double final_prediction = initial_prediction;
    for (size_t m = 0; m < ensemble.size(); ++m) {
        final_prediction += learning_rate * predict_single_tree(ensemble[m], x_i);
    }
}

```

```
    return final_prediction;  
}
```

The final prediction reconstructs $F_{M(x)} = F_0 + \nu \sum_{m=1}^M h_{m(x)}$ exactly.